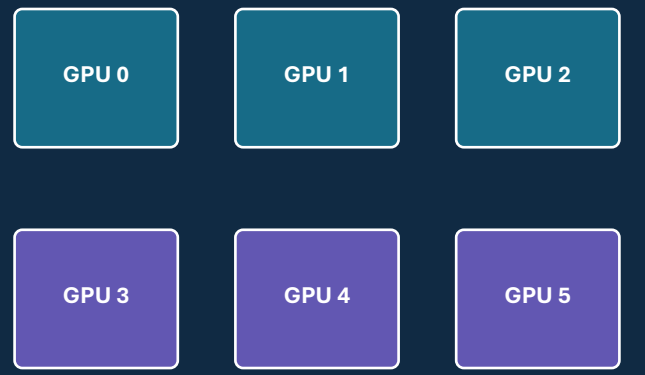
Scaling AI

Training Models Too Large for One GPU

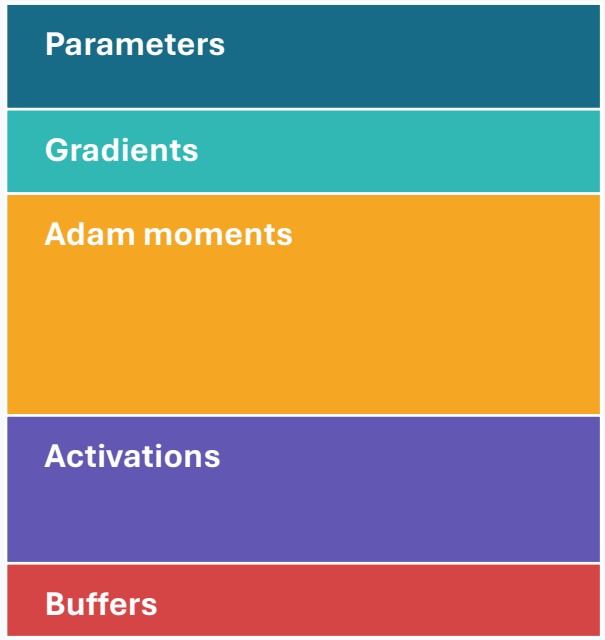


MODEL > GPU MEMORY

Dong Li
UC Merced

The memory wall

Training memory is far more than the parameter file.



80 GB limit

Replication

More data-parallel GPUs still store the full replica.

Partitioning

Split computation, depth, or training states.

Design question

How much communication buys each GB of memory?

01

Four dimensions of parallelism

The partitioned object determines the communication pattern.

Parallelism taxonomy

What is split? What is replicated? What moves?

DATA

SPLITS

examples

REPLICATES

full model

MOVES

gradients

TENSOR

SPLITS

matrices

REPLICATES

layer graph

MOVES

partial tensors

PIPELINE

SPLITS

layer groups

REPLICATES

stage weights

MOVES

activations

ZeRO / FSDP

SPLITS

model states

REPLICATES

compute schedule

MOVES

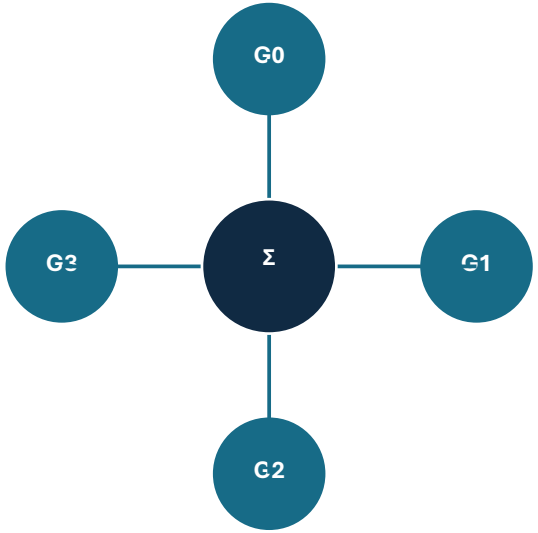
weights + gradients

THINK Which dimension primarily adds throughput? Which dimensions add capacity?

Collective communication diagrams

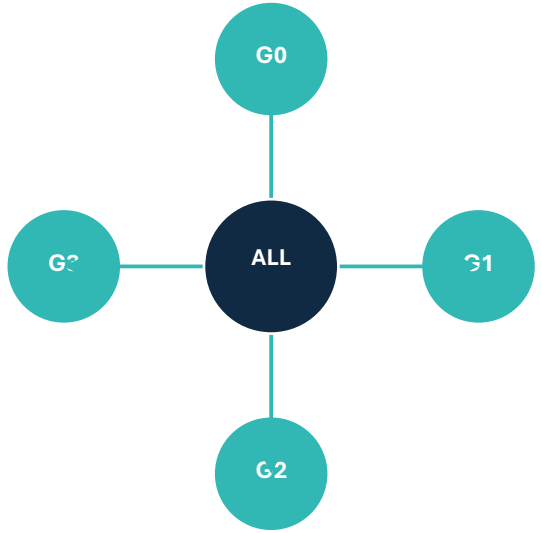
Training performance depends on where and how tensors move.

AllReduce



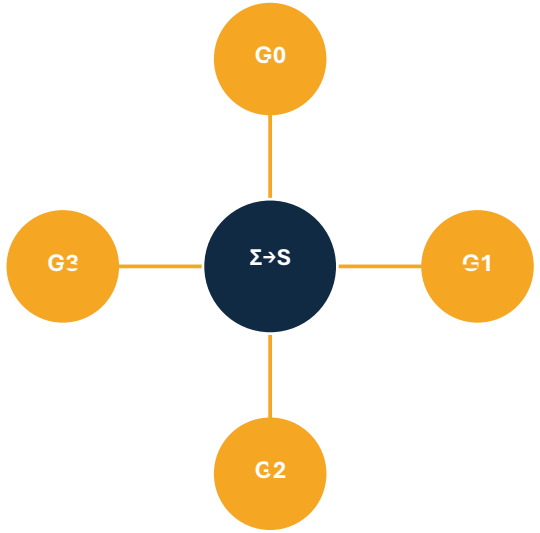
sum, then replicate

AllGather



assemble all shards

ReduceScatter



sum, then shard

THINK Match these to DDP, ZeRO parameter gathering, and ZeRO gradient sharding.

02

Tensor parallelism

Split the math inside a layer.

Column-wise tensor parallelism

Split output features: $W = [W_1 \mid W_2]$.

Result

$Y = [XW_1 \mid XW_2]$

Communication

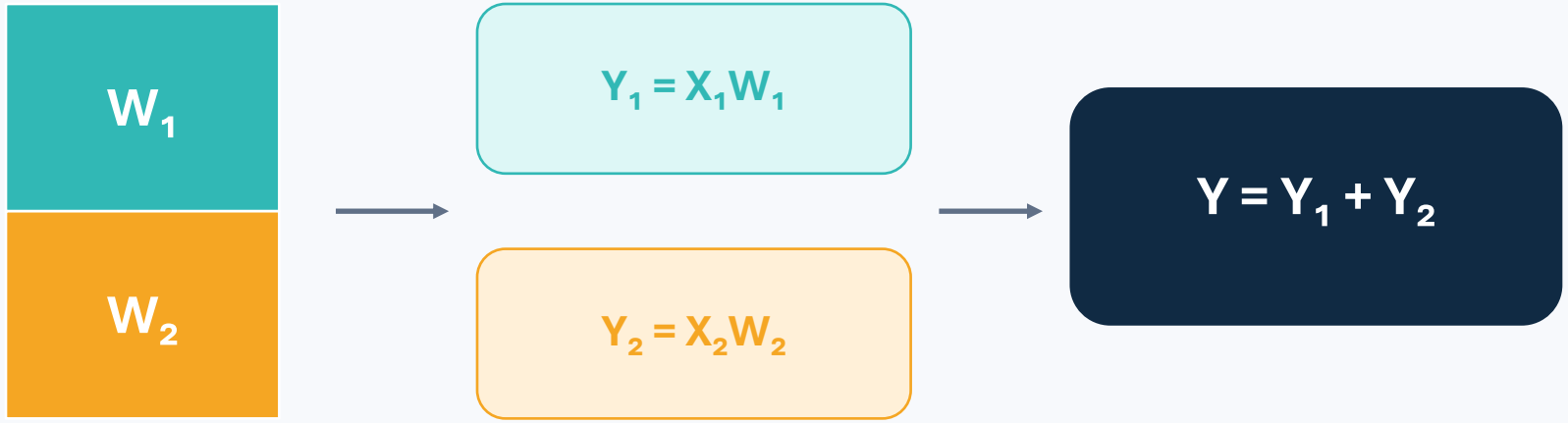
No sum at this boundary.

THINK Why can each GPU compute its output columns independently?

Worked example: row-wise tensor parallelism

Split input features and sum partial products.

$X = [X_1 \mid X_2]$ $W =$



GPU 0

computes X_1W_1

GPU 1

computes X_2W_2

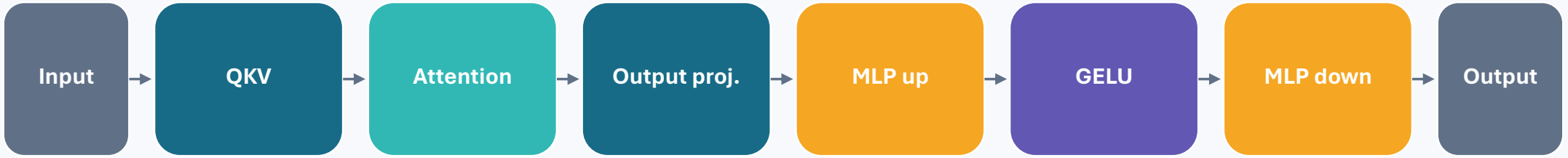
AllReduce

adds the partial outputs

THINK Column split concatenates; row split reduces.

Tensor parallelism inside a transformer block

Keep tensors sharded across adjacent projections when possible.



Column-parallel region

split heads or output features

Row-parallel boundary

sum partial outputs only when needed

THINK Why does TP prefer the fastest interconnect?

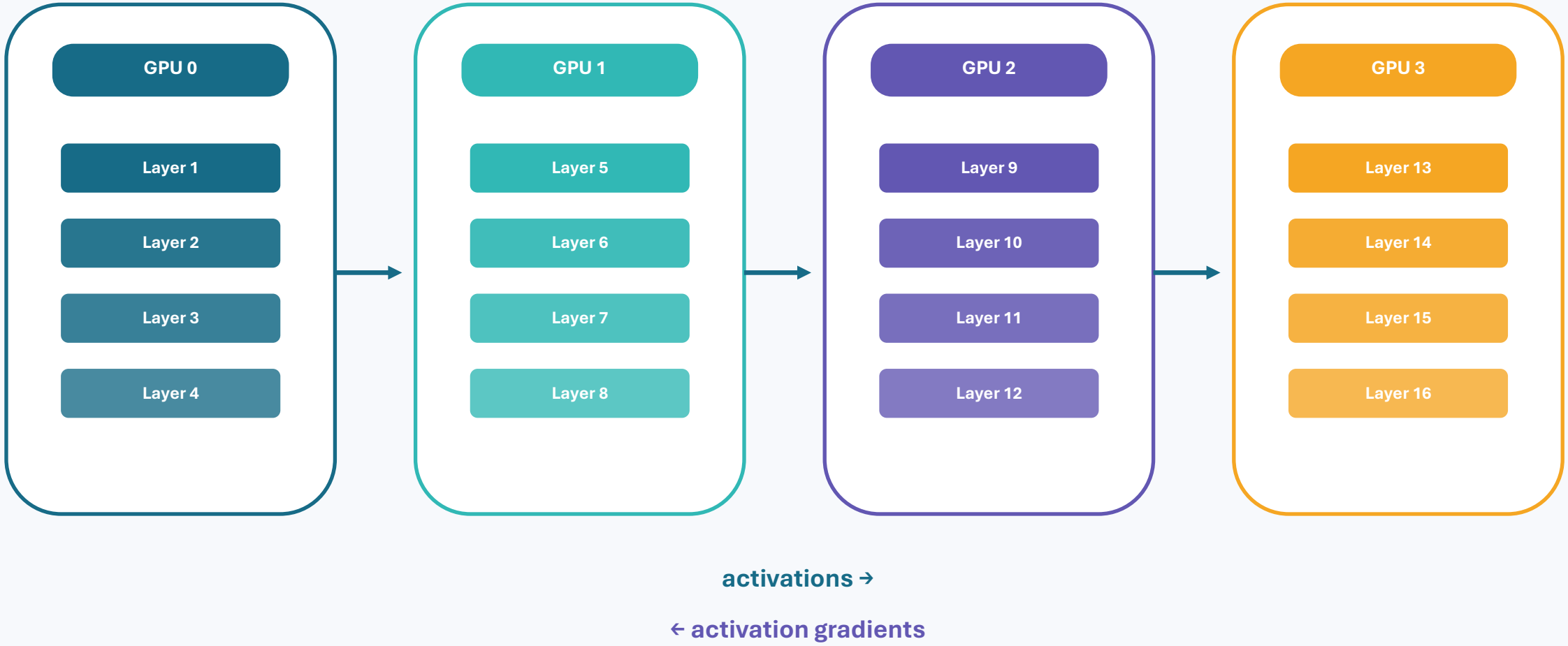
03

Pipeline parallelism

Split model depth and schedule microbatches.

Pipeline parallelism: split the depth

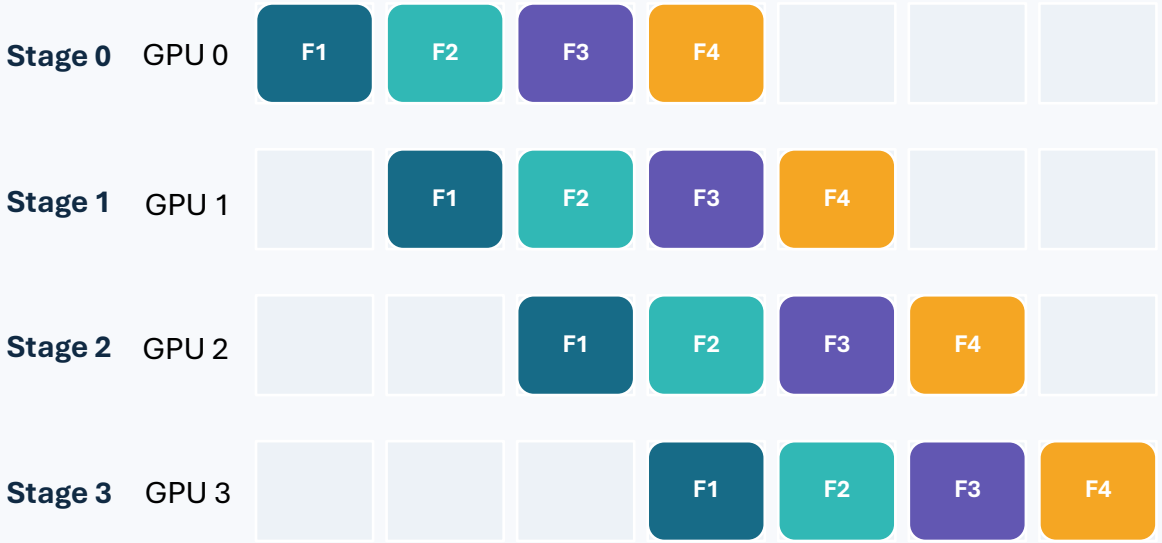
Stages exchange activations forward and activation gradients backward.



THINK The slowest stage determines pipeline throughput.

Pipeline schedule diagram

Four stages and four forward microbatches. Empty cells are bubbles.



Bubble fraction

$(p - 1) / (m + p - 1)$

p

pipeline stages

m

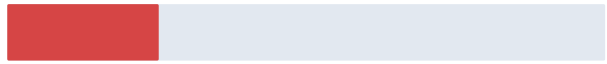
microbatches

THINK More microbatches amortize fill and drain.

Worked example: pipeline utilization

For p = 4 stages, utilization = m / (m + p − 1).

m = 1	1/4	=	25.0%
m = 4	4/7	=	57.1%
m = 16	16/19	=	84.2%



Interpretation

The bubble is fixed at p − 1 stage-times, but becomes a smaller fraction of a longer schedule.

THINK Why can very large m still be undesirable?

04

ZeRO and state sharding

Remove redundant copies across data-parallel workers.

ZeRO stages

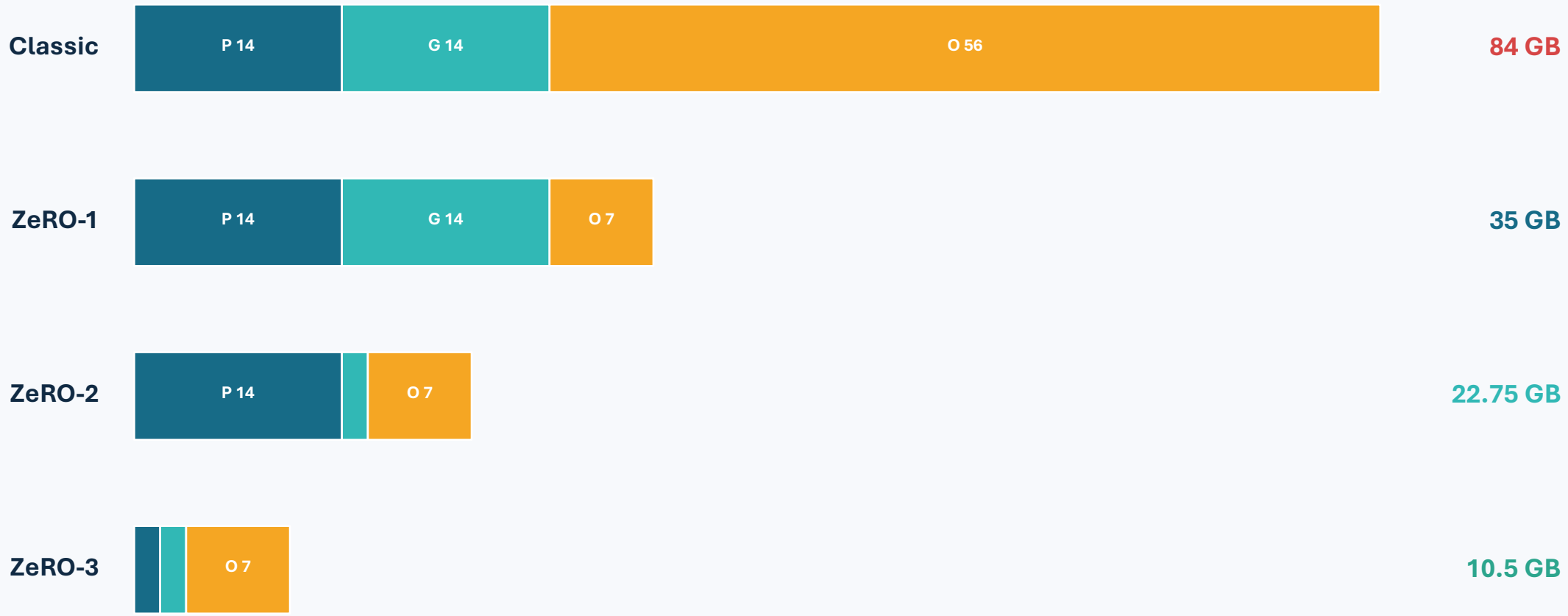
Progressively shard optimizer states, gradients, and parameters.

Classic DP	P G O	all replicated
ZeRO-1	P G O/DP	optimizer states
ZeRO-2	P G/DP O/DP	+ gradients
ZeRO-3	P/DP G/DP O/DP	+ parameters

THINK ZeRO-3 saves the most memory and gathers parameters during execution.

Worked example: ZeRO model-state memory

7B model, P=14 GB, G=14 GB, Adam moments O=56 GB, DP=8.



Excludes activations, master weights, temporary buffers, and fragmentation

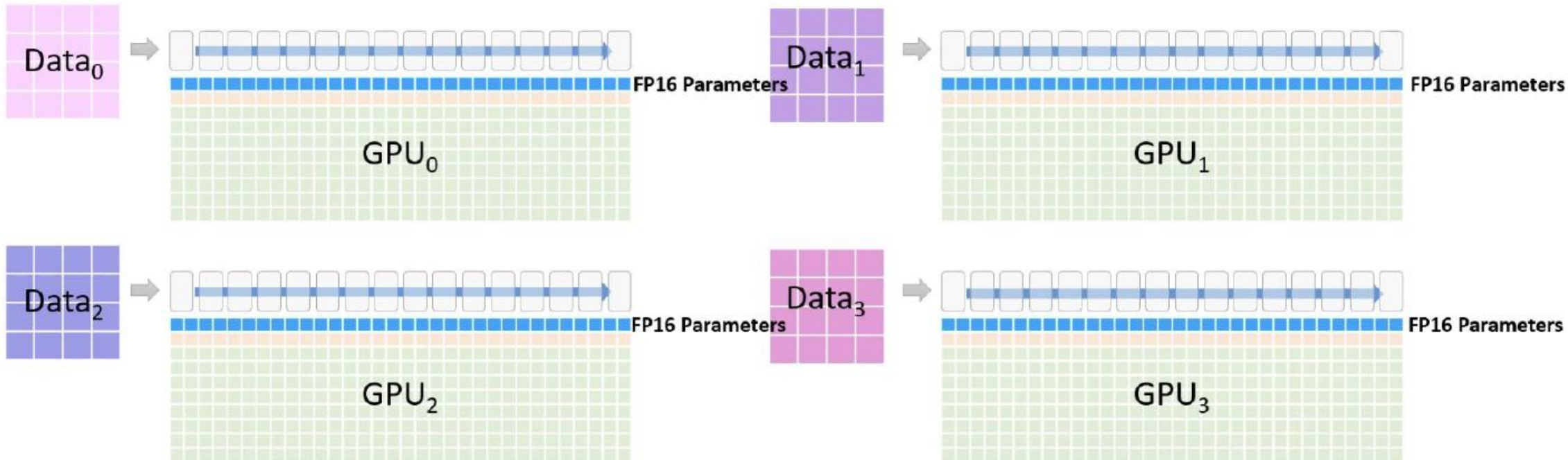
THINK ZeRO-1 is already below 40 GB in this simplified accounting.

ZeRO

Cells under a transformer layer represent its GPU memory:

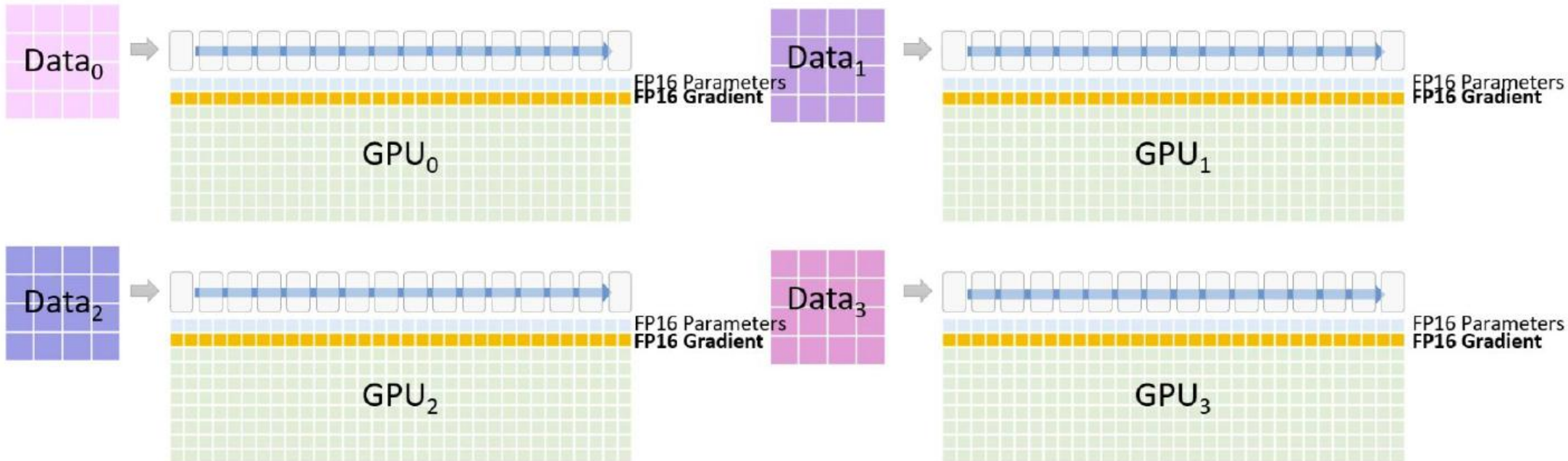
ZeRO

Blue cells represent model **parameters**:



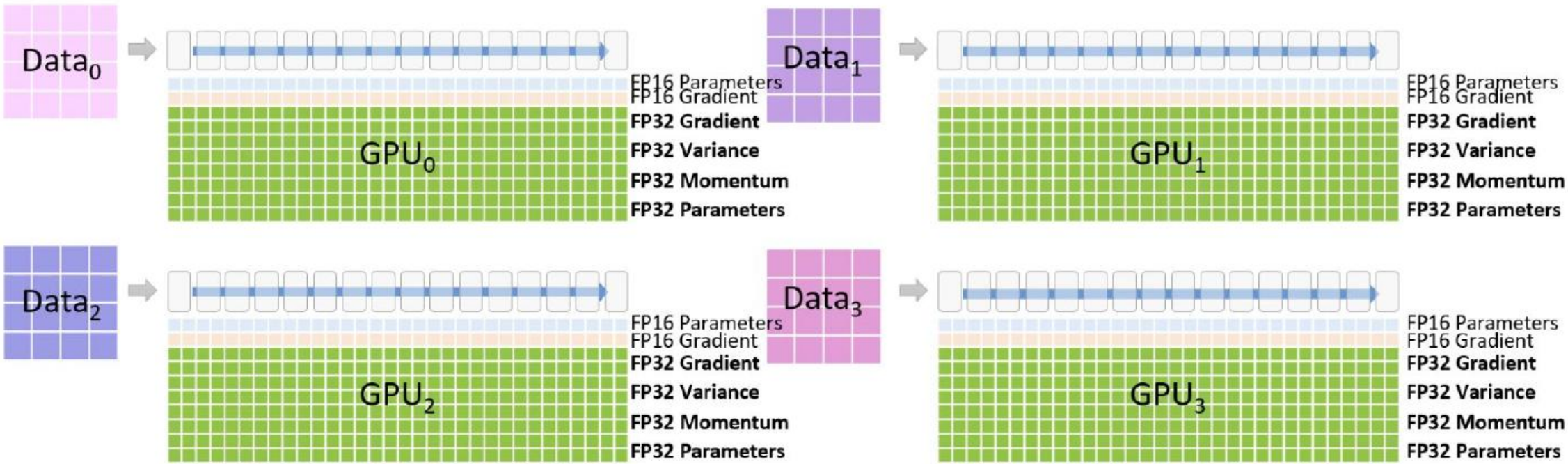
ZeRO

Orange cells represent **gradients**:



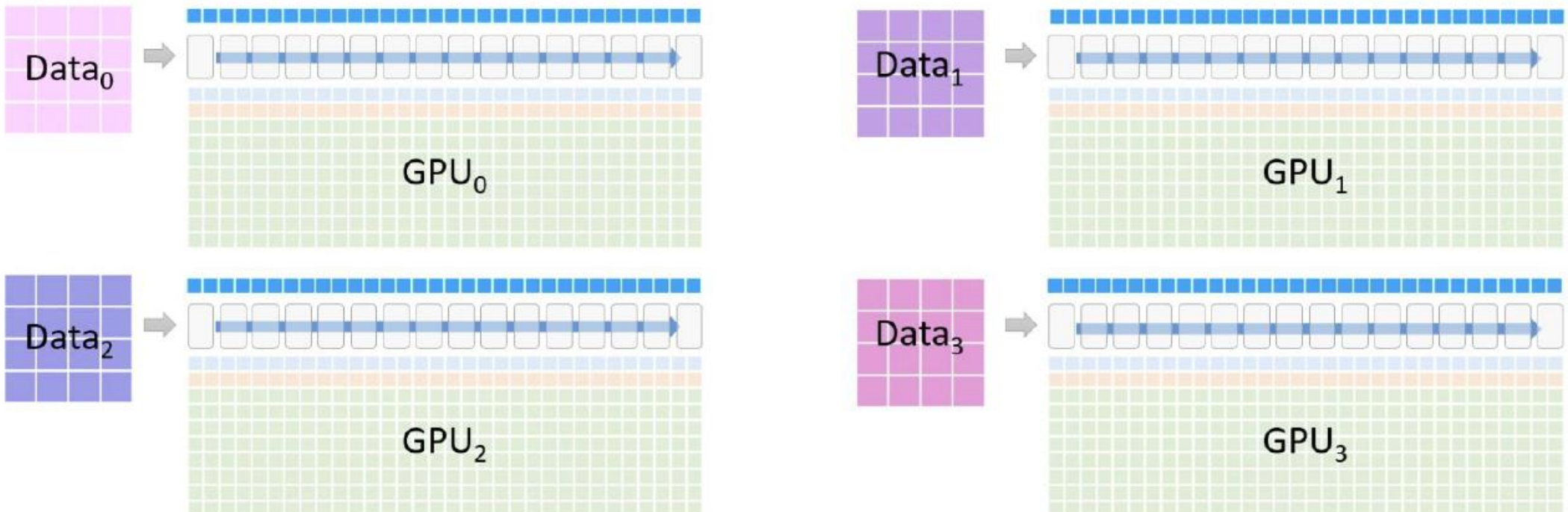
ZeRO

Green cells represent **optimizer state** (including temp. space for updated parameters):



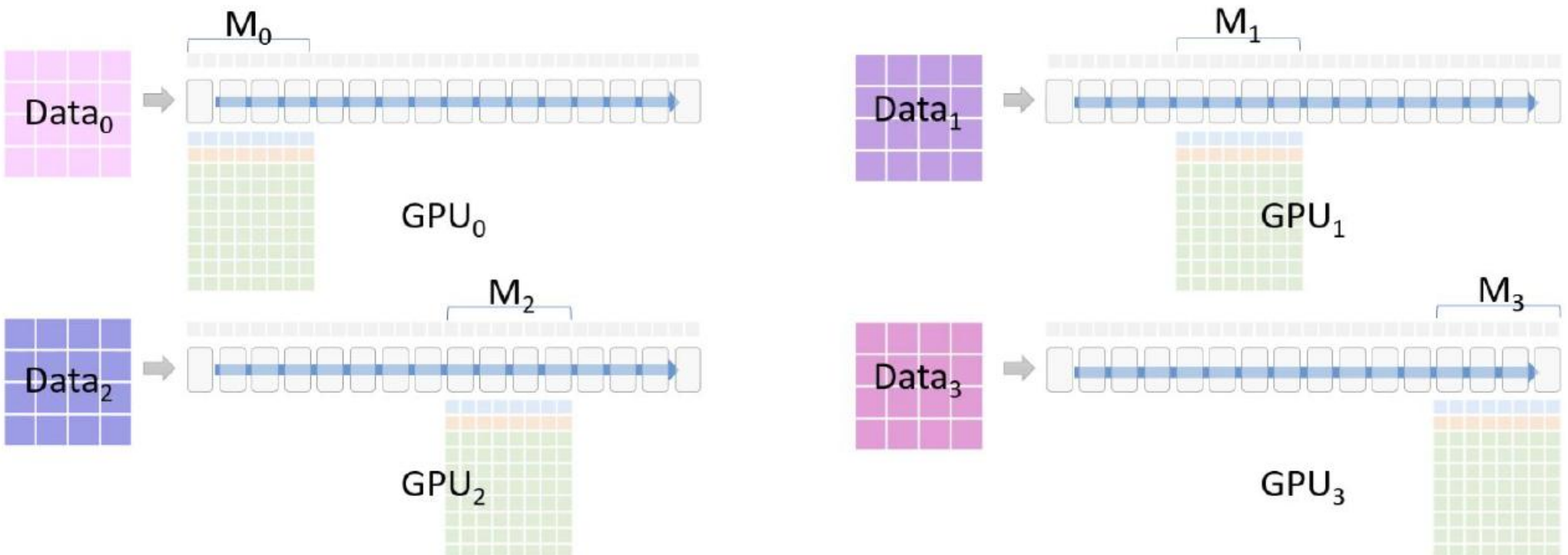
ZeRO

Blue cells along the top represent **activations**:



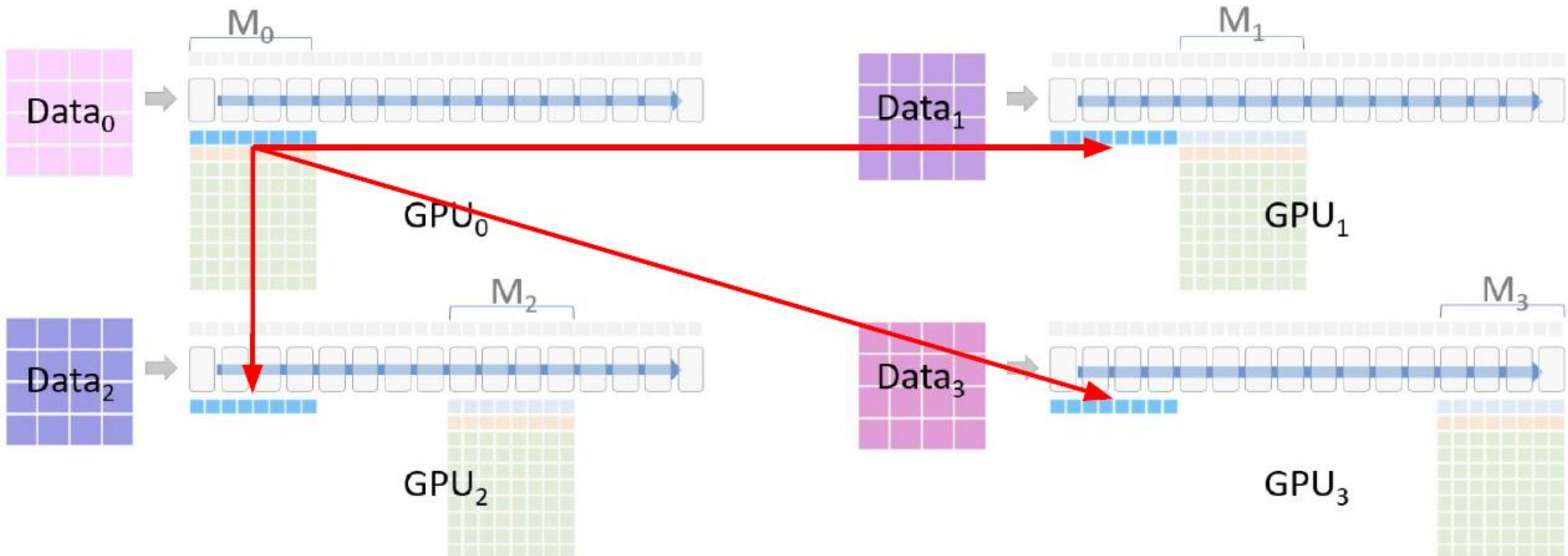
ZeRO

Model is **vertically partitioned** across each device:



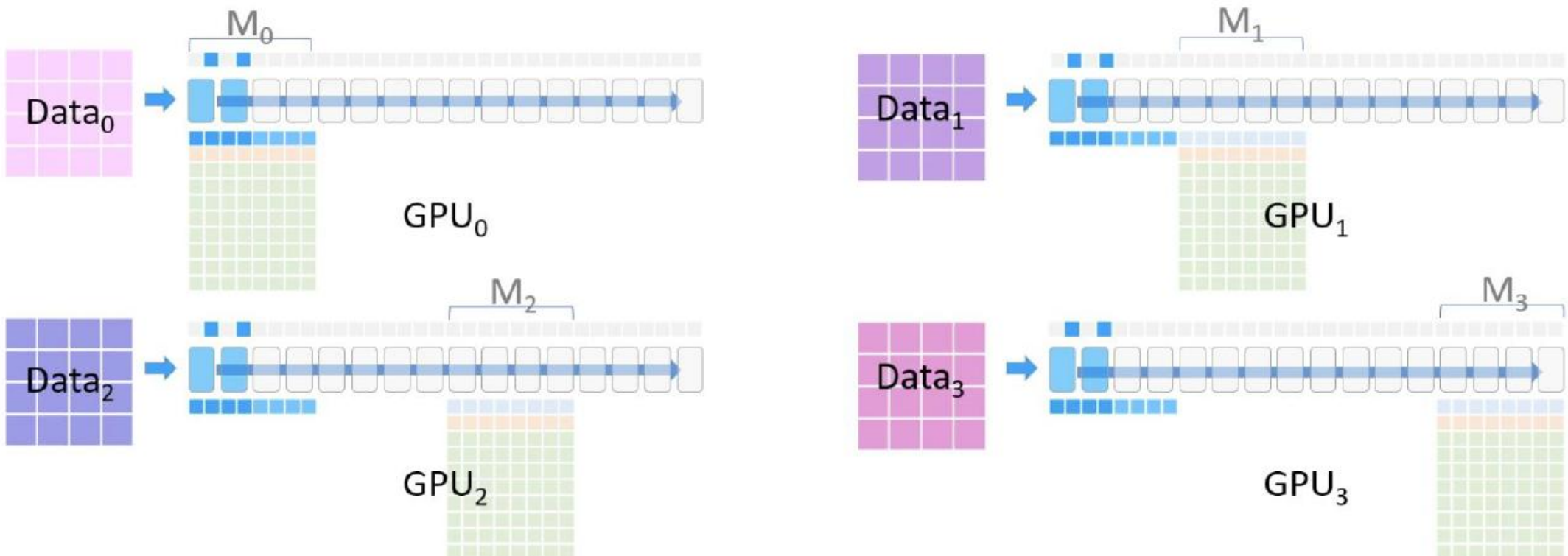
ZeRO

GPU_0 **broadcasts** its model partition:



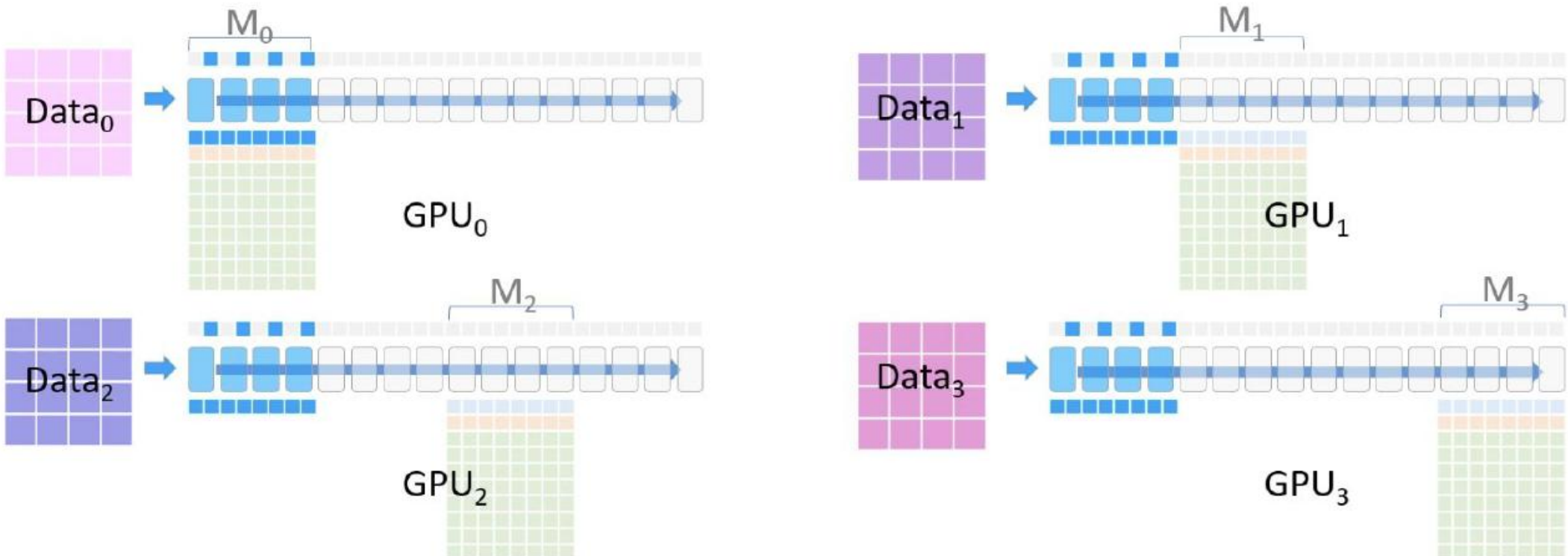
ZeRO

Each device evaluates M_0 on its own data:



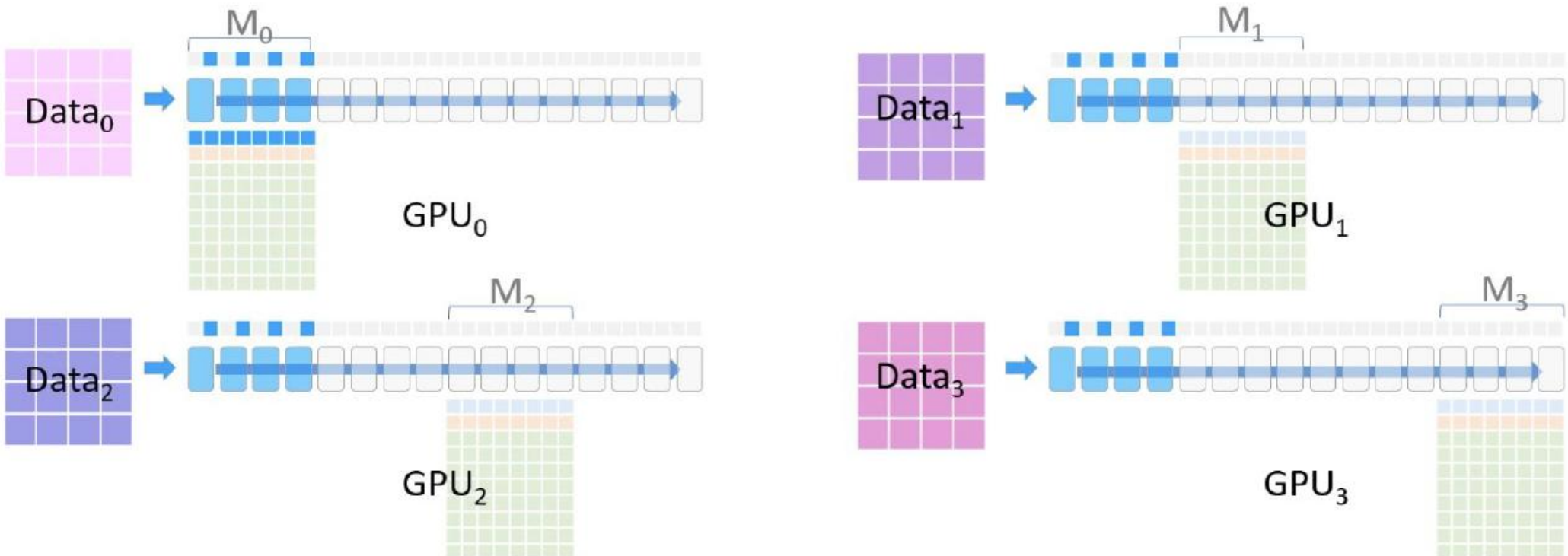
ZeRO

Each device evaluates M_0 on its own data:



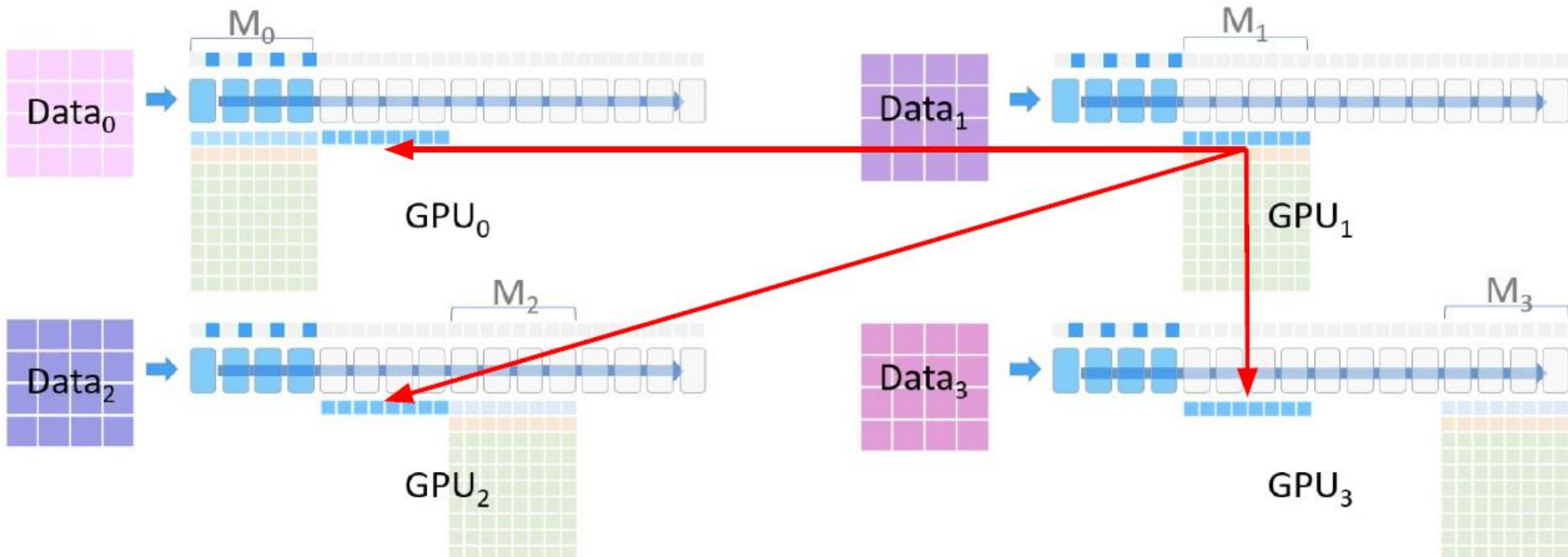
ZeRO

After the forward pass on M_0, all devices except GPU_0 delete M_0:



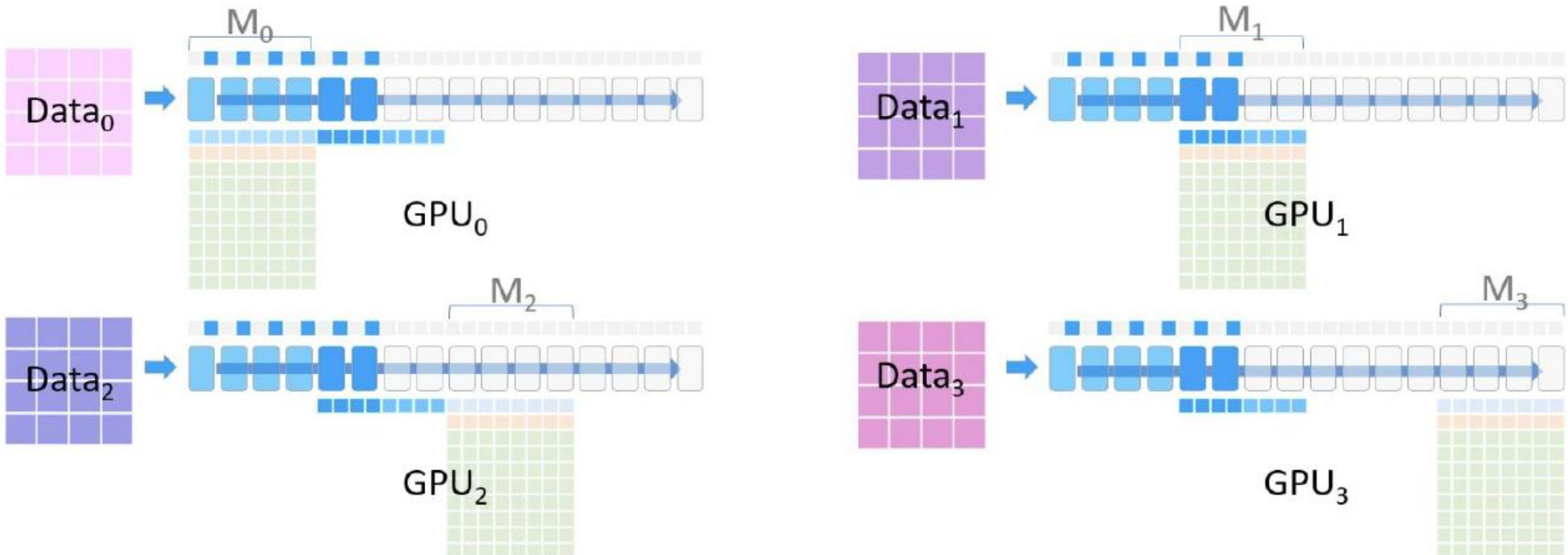
ZeRO

GPU_1 **broadcasts** its model partition:



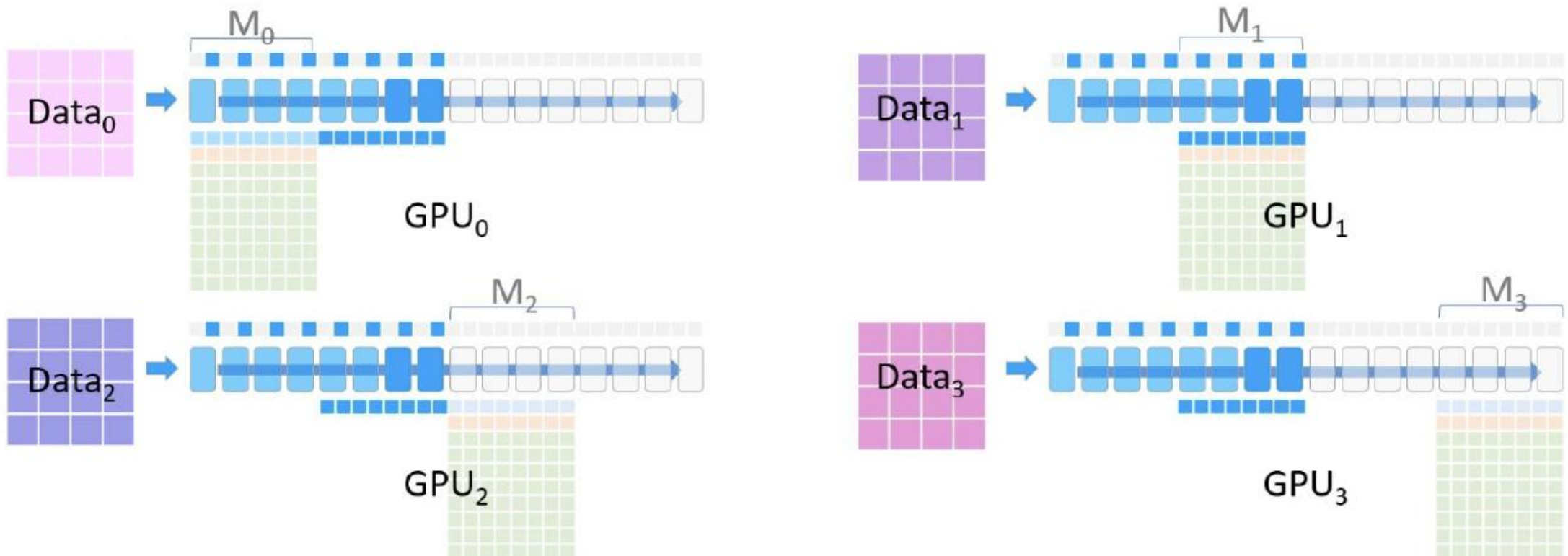
ZeRO

The forward pass continues on M_1:



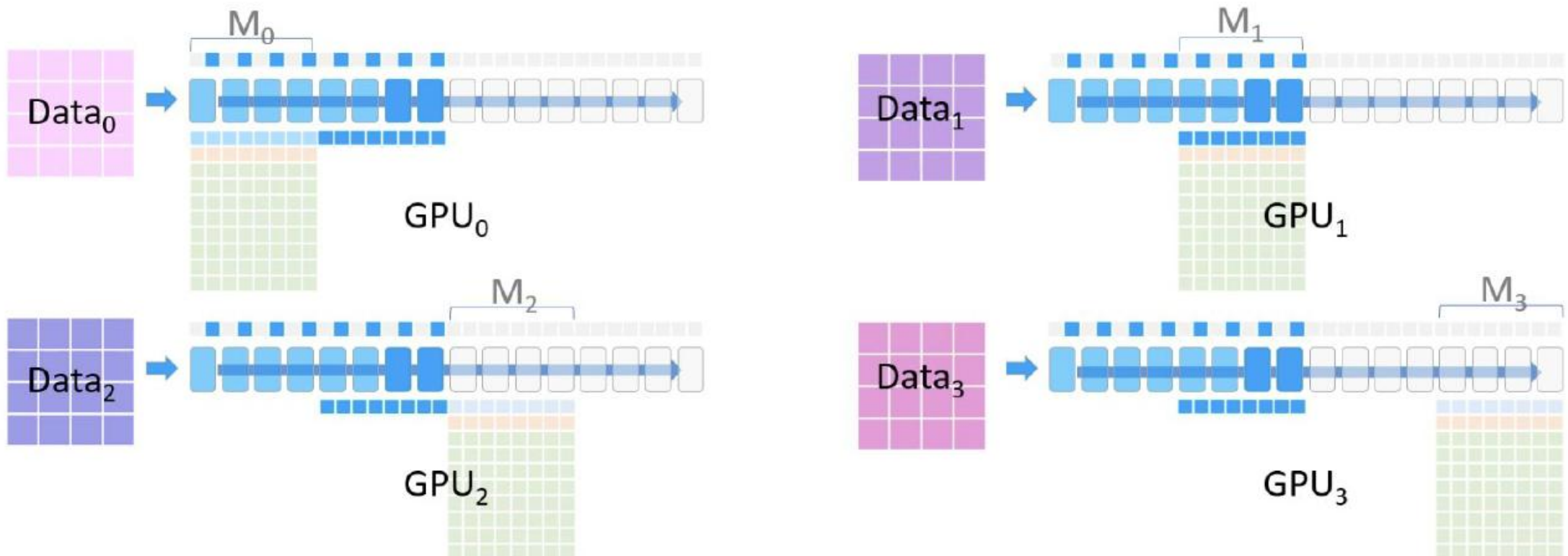
ZeRO

The forward pass continues on M_1:



ZeRO

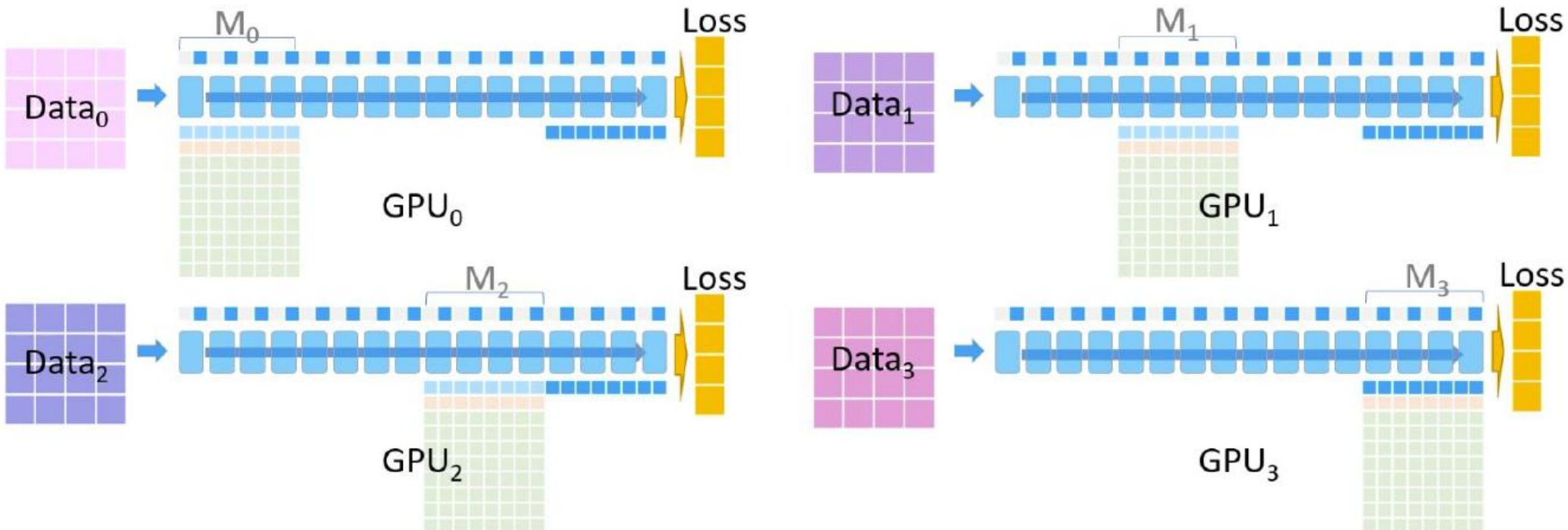
The forward pass continues on M_1:



We continue the similar process for M2 and M3

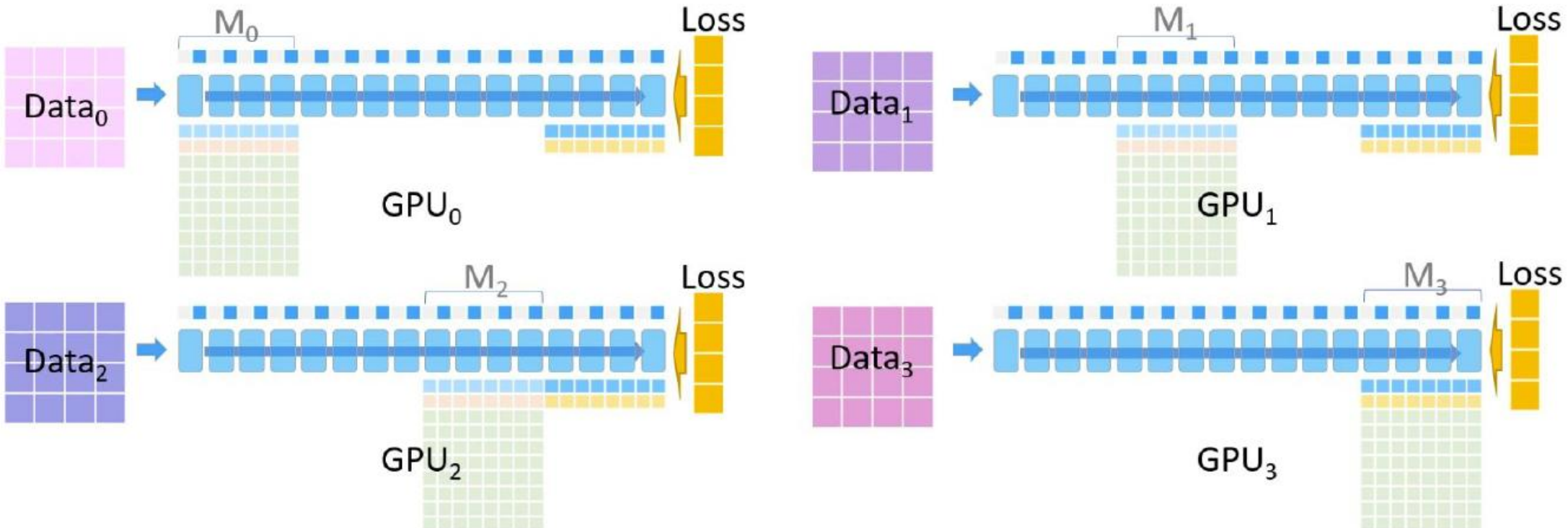
ZeRO

The forward pass is complete. In parallel, each device calculates the loss for its data:



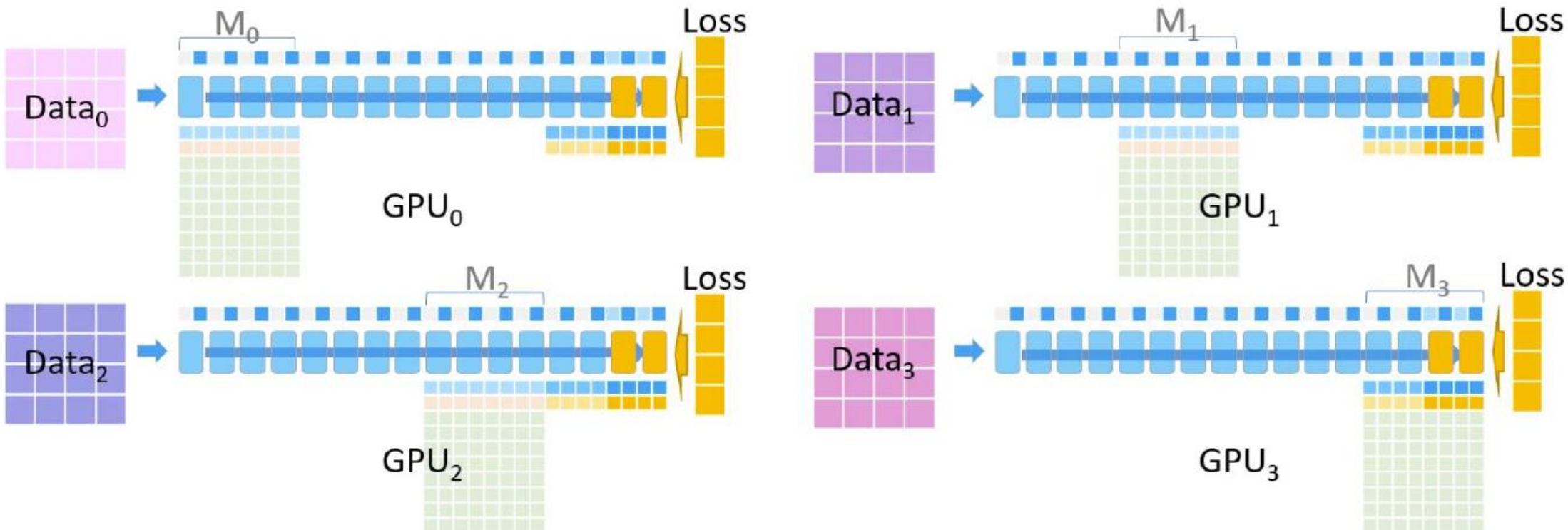
ZeRO

The backward pass starts. All devices allocate space for gradients (shown in pale yellow):



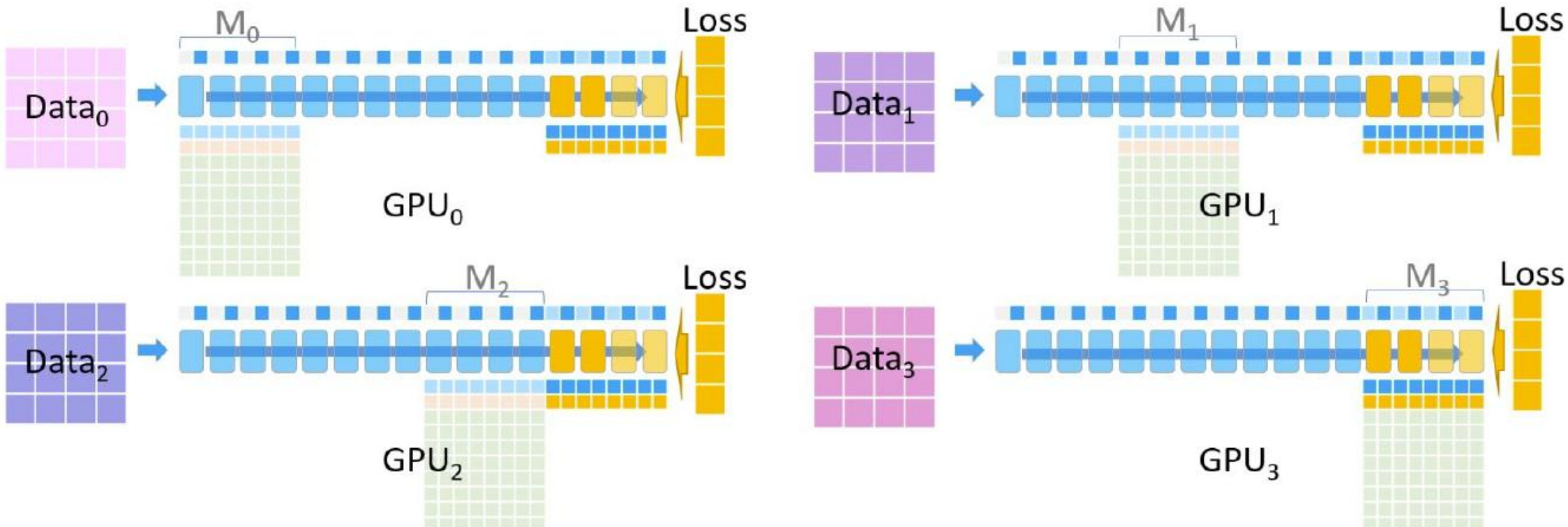
ZeRO

The backward pass proceeds on M_3:



ZeRO

The backward pass proceeds on M_3:

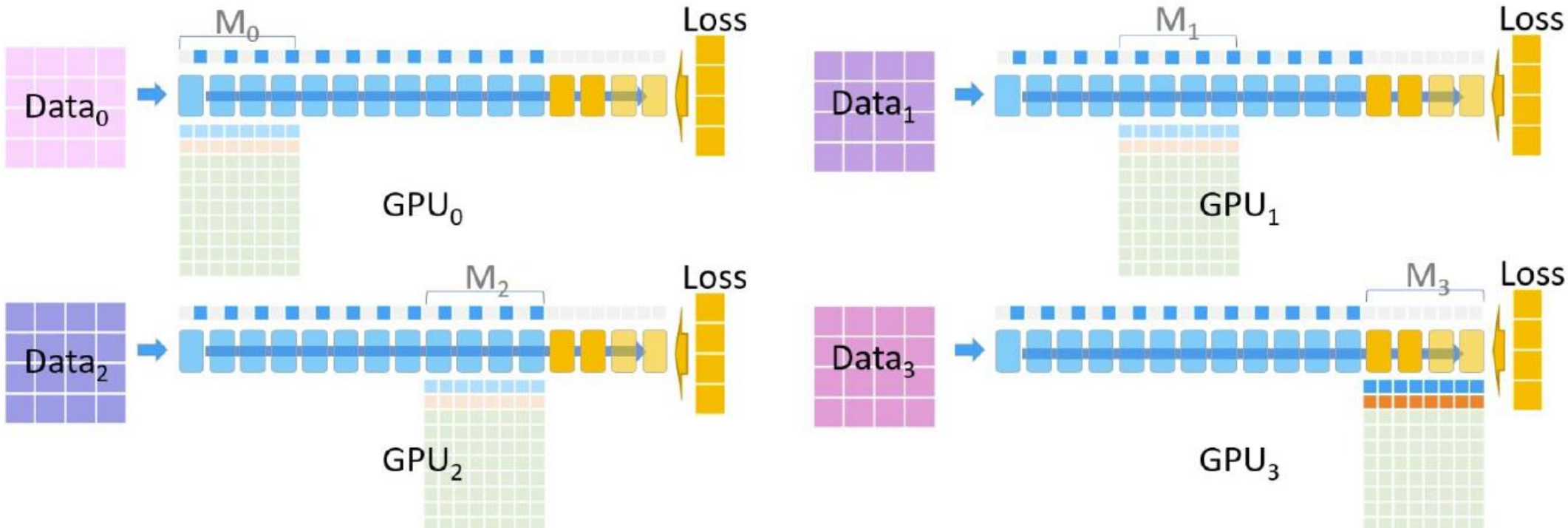


ZeRO

GPU_3 performs a **reduce** operation, holding the gradients for M_3 across all data:

ZeRO

All devices except GPU_3 delete M_3 and its gradients, and all devices delete activations for M_3:

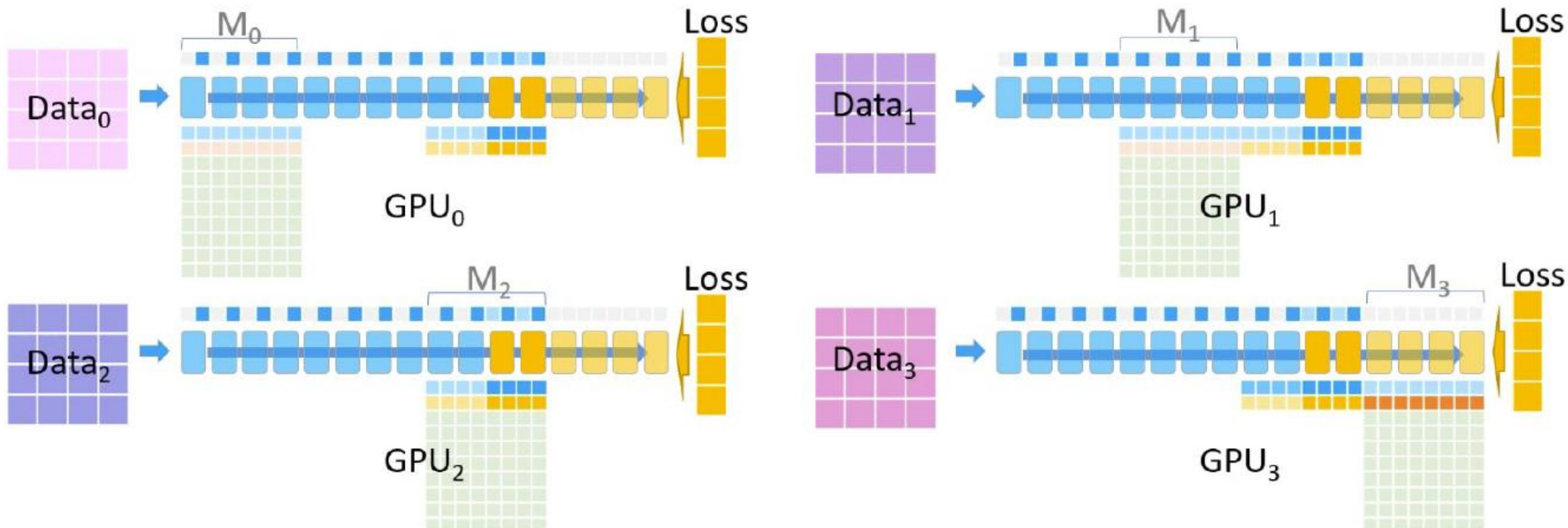


ZeRO

GPU_2 **broadcasts** its model partition:

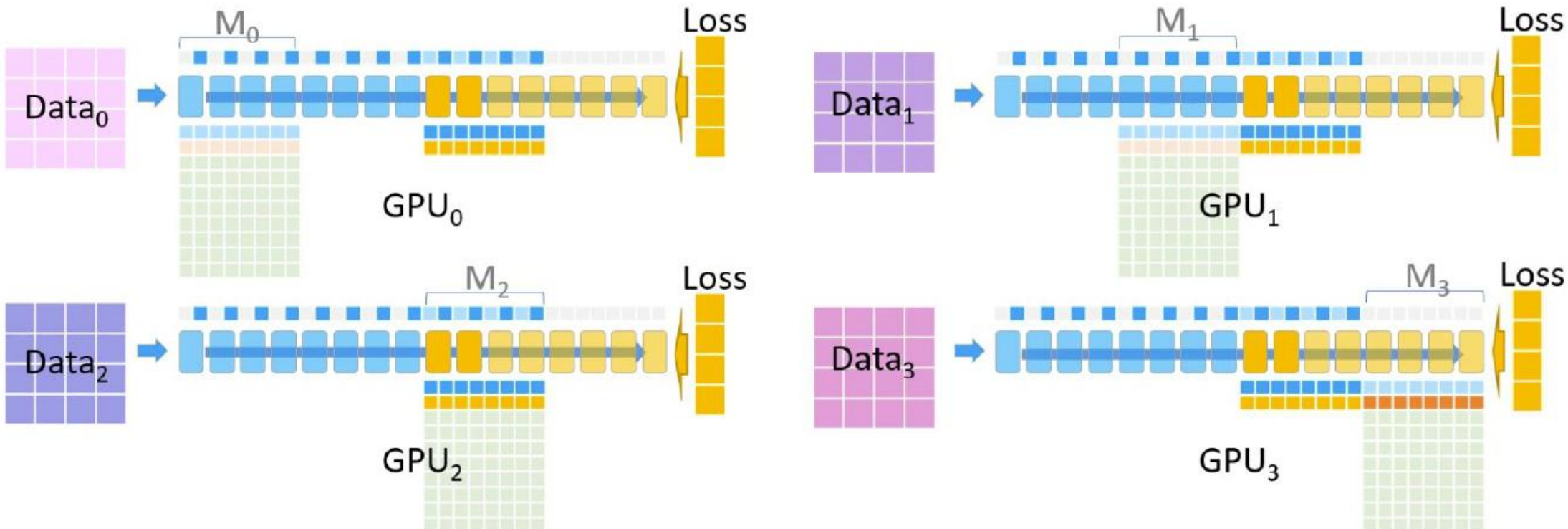
ZeRO

The backward pass continues on M_2:



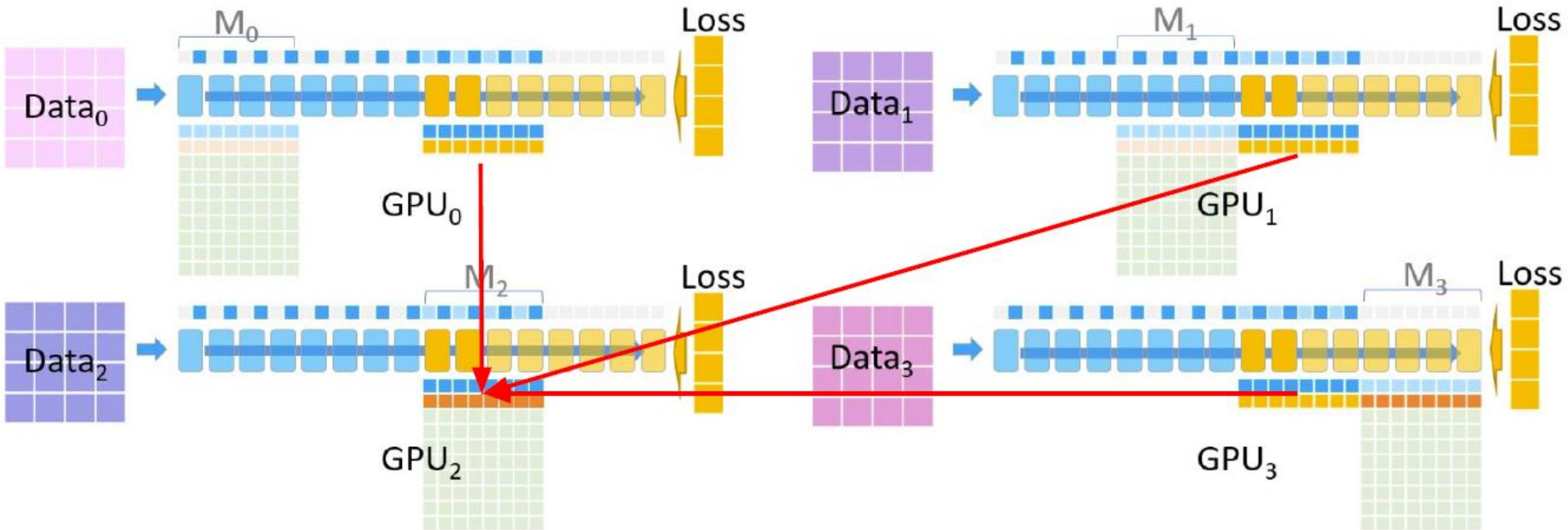
ZeRO

The backward pass continues on M_2:



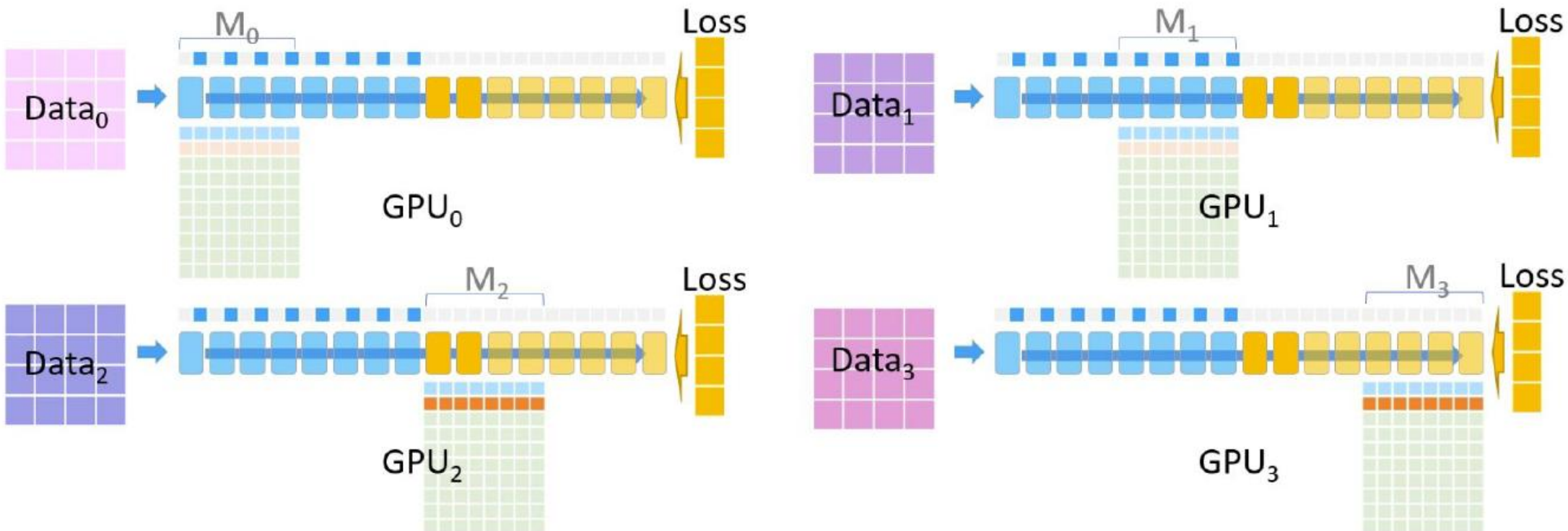
ZeRO

GPU_2 performs a **reduce** operation, holding the gradients for M_2 across all data:



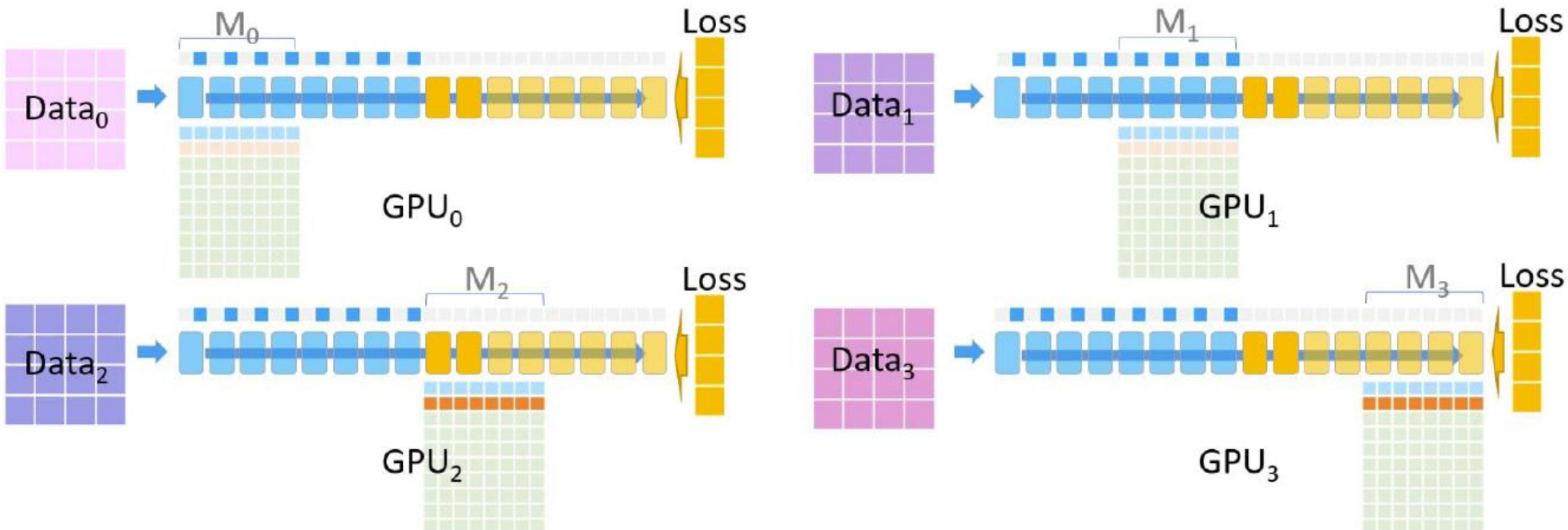
ZeRO

All devices except GPU_2 delete M_2 and its gradients, and all devices delete activations for M_2:



ZeRO

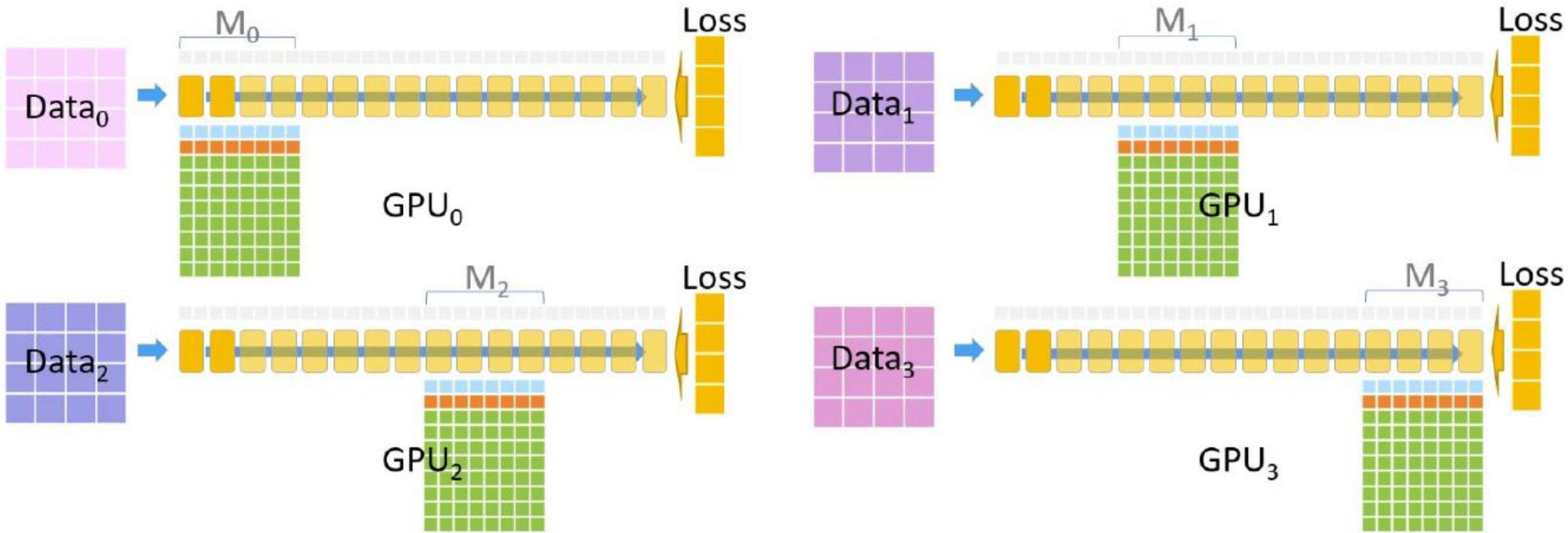
All devices except GPU_2 delete M_2 and its gradients, and all devices delete activations for M_2:



We continue the similar process for M0 and M1

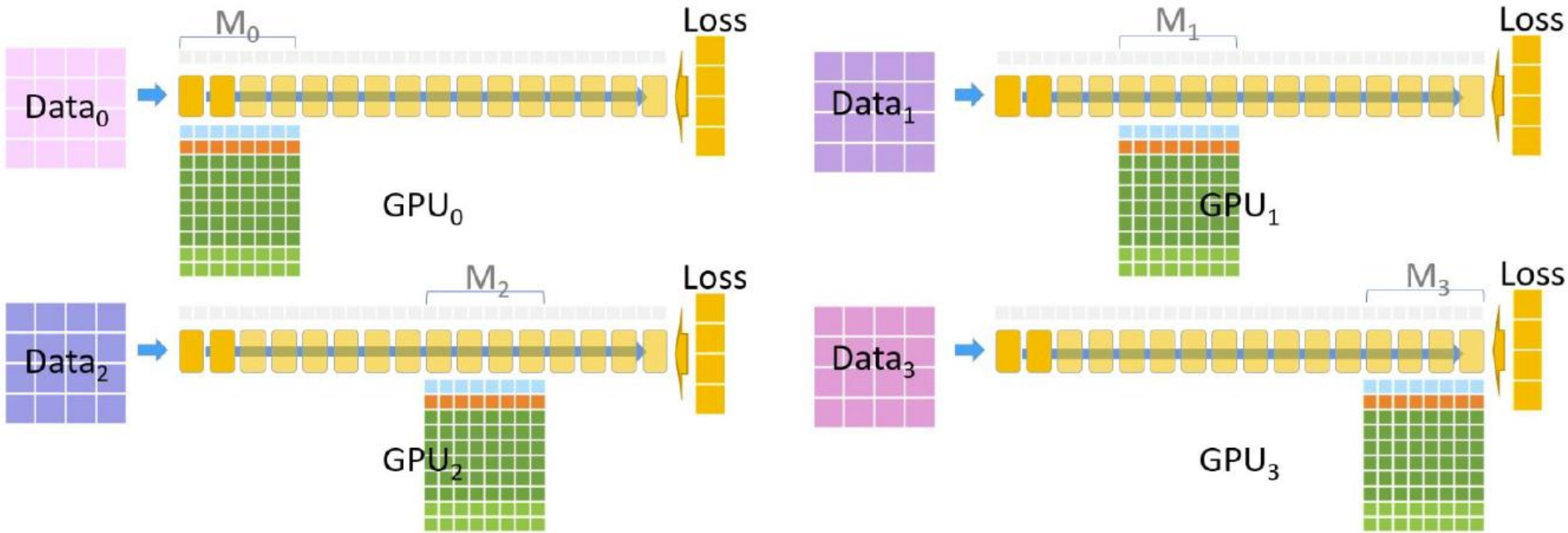
ZeRO

The backwards pass is complete. In parallel, each device performs the optimization step:



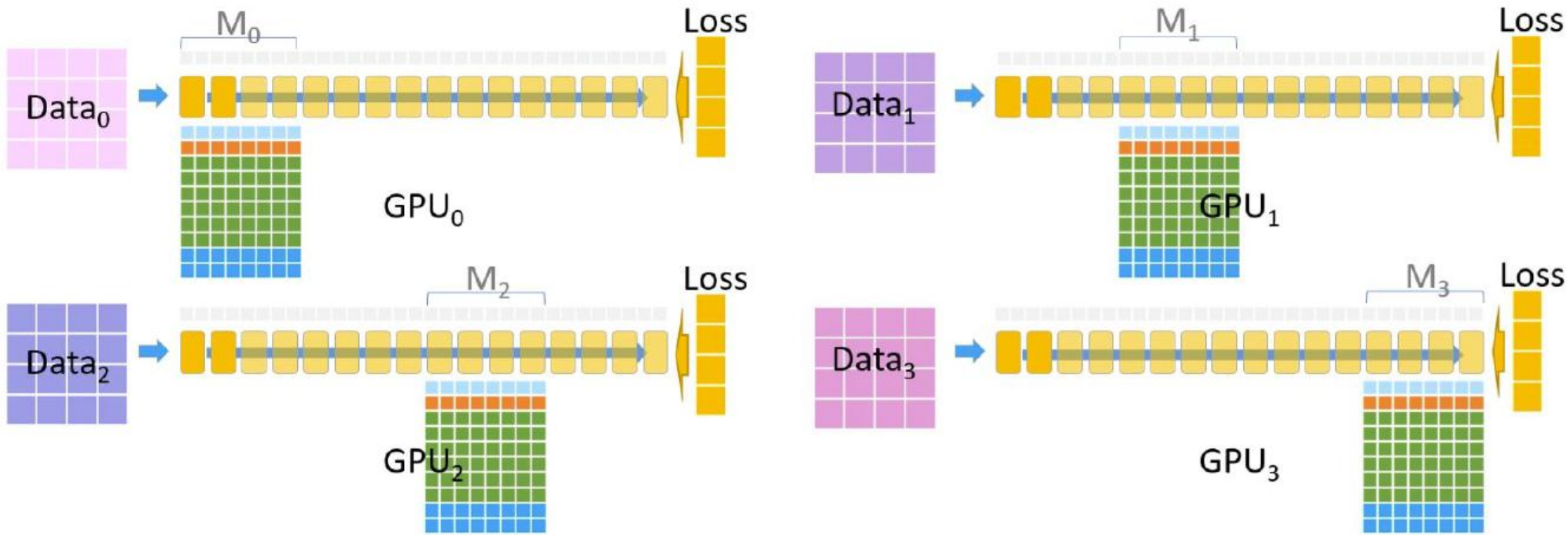
ZeRO

The optimizer runs:



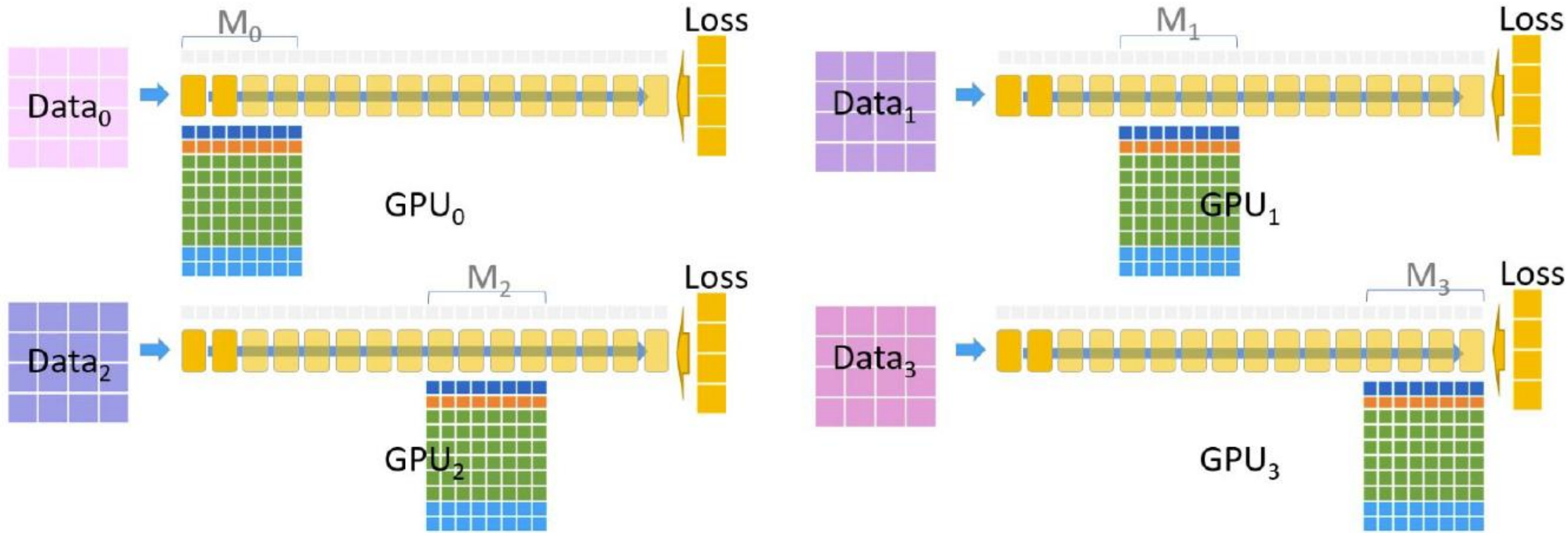
ZeRO

The updated model parameters are output as FP32:



ZeRO

The updated model parameters are converted to FP16 and replace the old parameters:



ZeRO

This method vertically partitions the model. Won't this cause the same issues as PP?

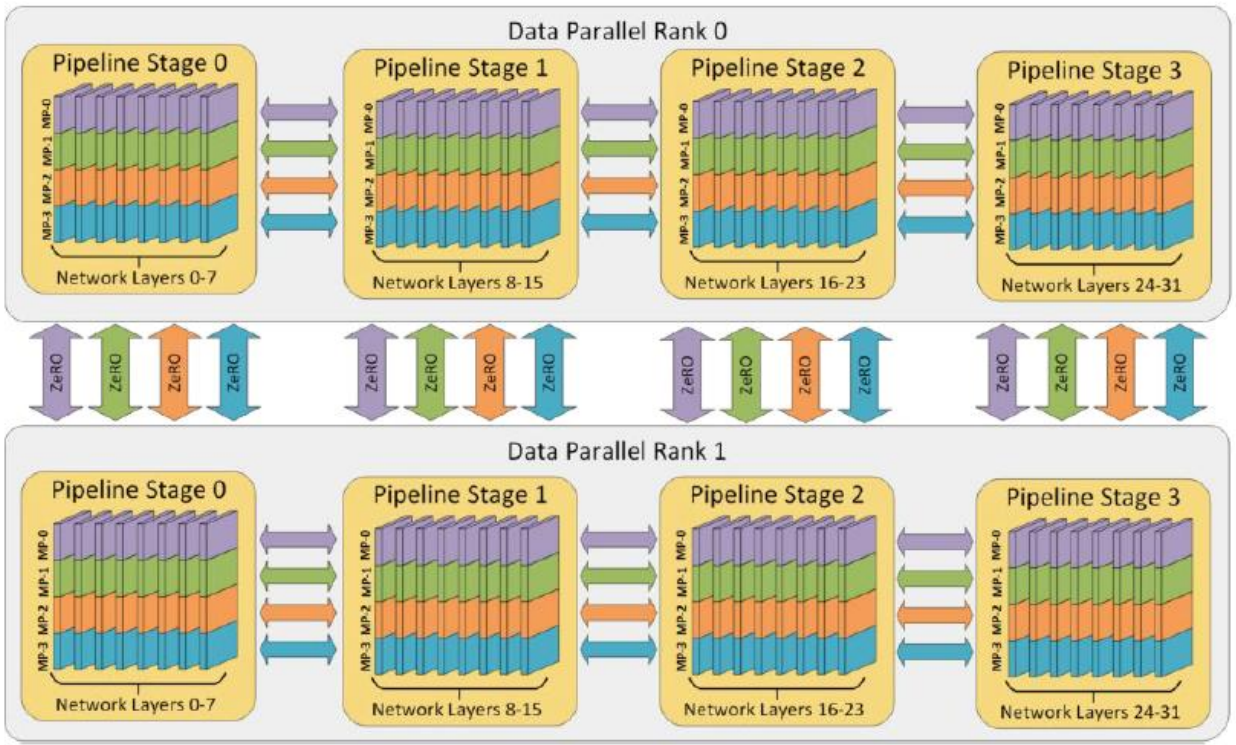
- Model partitions are broadcasted between devices
- Each device actually evaluates the entire model over the course of a forward/backward pass

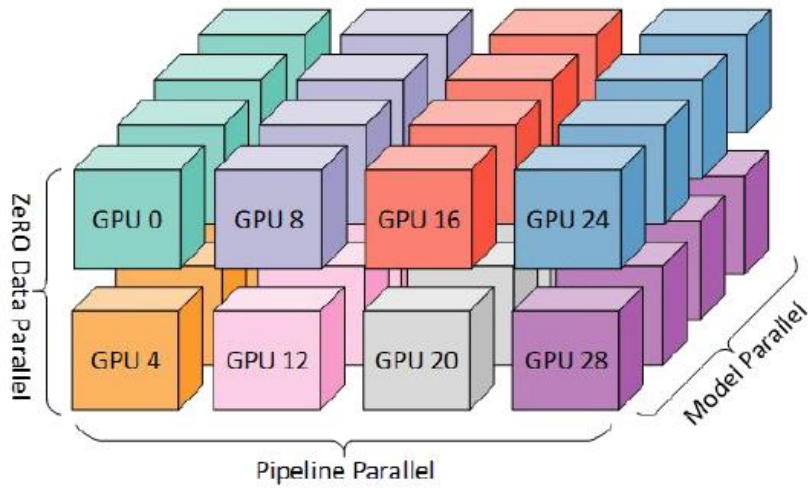
That seems like a lot of communication. Won't this cause the same issues as MP?

- It is a lot of communication, but…
- MP communicates **activations**
- ZeRO communicates **parameters**
- Sending parameters can be done async. with forward/backward pass calculations

DeepSpeed: ZeRO + MP + PP

Combines ZeRO stages with other parallelization methods to optimize hardware/network performance:





https://microsoft.com/research/blog/deepspeed-extreme-scale-model-training-for-everyone/

https://github.com/microsoft/DeepSpeed

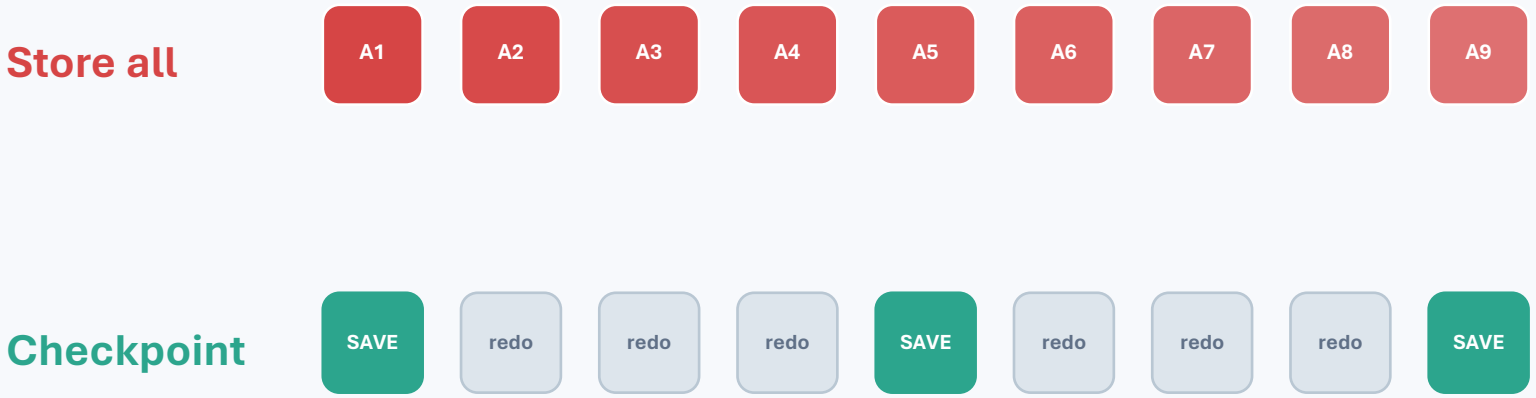
05

Efficient transformer training

Trade memory, compute, precision, and data movement.

Activation checkpointing diagram

Store selected boundaries; recompute missing intermediates during backward.



Memory

fewer saved activations

Compute

extra forward recomputation

Use when

activations limit capacity

THINK Checkpointing improves capacity, not raw throughput.

Worked example: activation scaling

Use a simplified proportional model before a detailed profiler.

Baseline

microbatch 4
sequence 2,048
activation estimate 24 GB

Double sequence

microbatch 4
sequence 4,096
linear estimate 48 GB

Halve microbatch

microbatch 2
sequence 4,096
linear estimate 24 GB

$$M_{act} \propto \text{microbatch} \times \text{sequence} \times \text{hidden size} \times \text{layers}$$

06

Hybrid parallel design

Compose dimensions, then map them to hardware topology.

3D parallelism arithmetic

Group sizes multiply to the total GPU count.

TOTAL GPUs = DP × TP × PP

DP = 4

four data replicas

TP = 4

four GPUs per layer

PP = 2

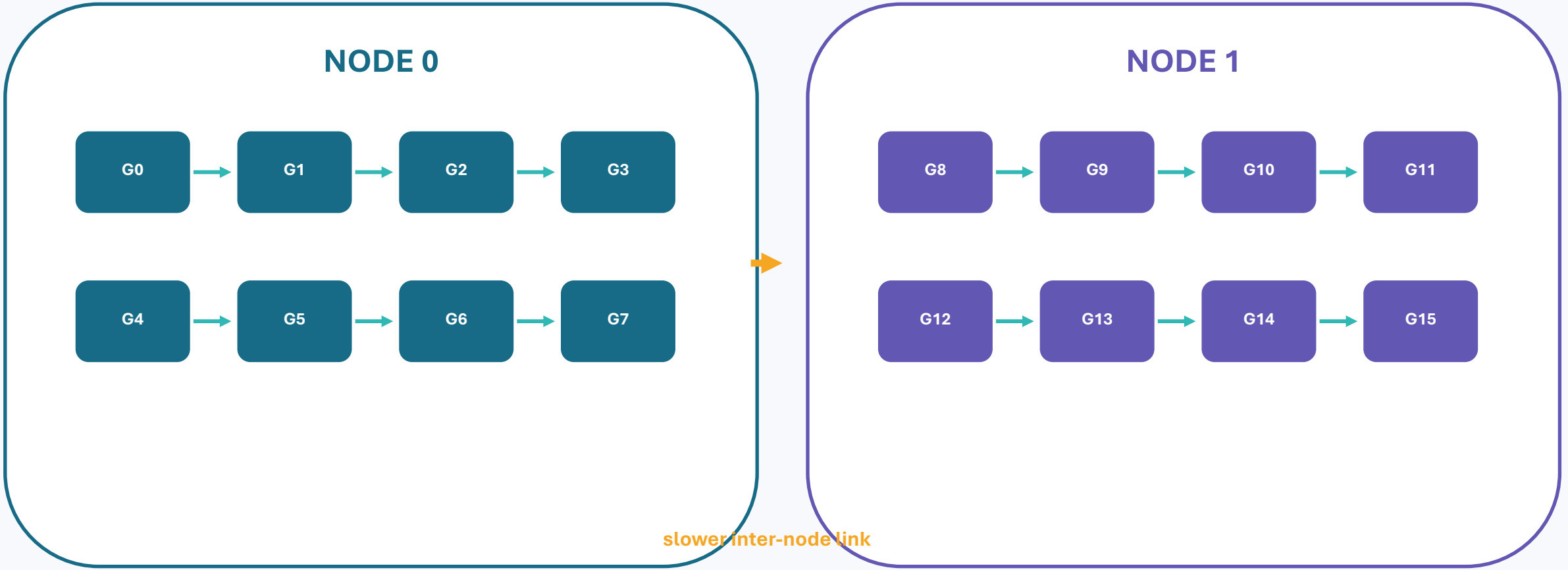
two depth stages

4 × 4 × 2 = 32 GPUs

THINK Which dimension should live on the fastest links?

Worked placement: two 8-GPU nodes

Map frequent communication inside a node; cross nodes less often.



Example: TP=8 within each node; DP or pipeline boundary crosses nodes.

Worked design: can a 30B model fit?

32 GPUs, 80 GB each, simplified 12 bytes/parameter model-state estimate.

1. Baseline

30B × 12 bytes = 360 GB
before activations

2. Split model

TP=4 and PP=2
360 ÷ 8 ≈ 45 GB

3. Add replicas

DP=4
4 × 4 × 2 = 32 GPUs

Approximate per-GPU accounting

45 GB model states
+ sharded/checkpointed activations
+ temporary buffers
+ safety headroom

What could invalidate this estimate?

12-byte assumption
uneven stage sizes
unsharded activations
communication workspaces

THINK A feasible paper design must still be measured on hardware.

Worked comparison: two 32-GPU plans

Both are feasible on paper, but stress different resources.

PLAN A

DP 8 × TP 4 × PP 1

Advantage

No pipeline bubble

Cost

More DP communication; model must fit with TP + ZeRO

PLAN B

DP 4 × TP 4 × PP 2

Advantage

More capacity from depth split

Cost

Pipeline bubble and stage balancing

THINK Choose using peak memory, link utilization, and pipeline idle time.

Lab challenge

Build a memory simulator and defend one configuration.

A Estimate

parameters, gradients, states, activations

B Model

DP, TP, PP, ZeRO stages

C Search

all DP × TP × PP factorizations

D Defend

memory, communication, bottleneck, limitation

THINK Compare at least two feasible strategies, not just one.

Five ideas to remember

1. Data parallelism adds throughput, not single-replica capacity.
2. Tensor parallelism splits layer math and communicates frequently.
3. Pipeline parallelism splits depth and must manage bubbles.
4. ZeRO removes redundant model-state memory.
5. **Efficient scaling balances compute, memory, communication, and utilization.**

Exit ticket: What bottleneck does your chosen technique remove, and what cost does it add?